Rootine Backend: Registration, Explained

A beginner-friendly guide to the account-creation flow, the folders it uses, the code line by line, and the programming foundations behind it.

Prepared from the current Rootine backend files and the registration tests performed during this session.

What you will learn

- How a registration request travels from an HTTP client through Express to the database.
- What routes, controllers, services, repositories, middleware, schemas, and database files
- do.
- What imports, Zod's z, TypeScript types, async/await, bcrypt, and Drizzle queries mean.
- What the registration test results proved, and what still needs attention.

The examples and descriptions below are based on the code currently in the project. This is a learning guide, not a claim that login or email verification is already complete.

1. The big picture

Imagine a small office that opens new customer accounts. A visitor submits a form. A receptionist checks that it is filled out correctly, a clerk handles the request, a policy desk checks the rules, a records desk talks to the database, and the office returns a result.

In this app, those roles are separated into code folders. Each part has one main responsibility. The separation makes it easier to understand, test, and change the system.

The registration request journey

- A client sends HTTP POST /auth/register with JSON account details.
- Express parses the JSON body in app.ts.
- The auth router matches /register, because app.ts mounted that router under /auth.
- Validation middleware checks the request against registerSchema.
- The register controller calls signUpService and sends its result as JSON.
- The signup service checks for an existing email, hashes the password, and asks
- UserRepository to create the account.
- UserRepository uses Drizzle to query or insert a row in the PostgreSQL users table.
- The response travels back to the client with an HTTP status such as 201 or 400.

A route analogy

Think of a building address: the app is the building, /auth is a department, and /register is a desk in that department. Together the full route is /auth/register. POST means the visitor is submitting information, rather than asking to view a page.

```
POST http://localhost:7000/auth/register
```

What the tests showed

- A valid request returned 201 and a success message. The duplicate request then returned 400,
- which confirms the account was saved and the duplicate check found it.
- A request without city returned 400 with a Zod validation issue. It was rejected before
- account creation.
- The displayed PowerShell response showed an empty-looking user value. The repository is
- intended to return the new user's ID, so verify that response payload before relying on it.
- The successful request wrote an account into the database. Keep that in mind; do not delete
- or reset the database to continue the lesson.

2. What the folders are for

src/routes - the directory and signposts

Routes define which HTTP method and URL lead to which handlers. The auth router contains POST /register. Middleware is listed before the controller, so Express runs validation first.
Real-life analogy: a signpost directs a visitor to the right counter.

`src/routes/auth-routes.ts`

src/controllers - the receptionist

A controller receives the Express request and chooses the HTTP response. The register controller passes req.body to the service, then uses the service's code and result to respond. It should not contain the database's detailed query logic.

`src/controllers/auth-controllers.ts`

src/services - the business procedure

A service coordinates what the application must do for a task. Signup checks for an existing account, hashes the password, creates the user, and returns a result. Analogy: the receptionist follows the office's account-opening procedure.

`src/services/auth/Sign-Up-Service.ts`

src/repositories - the records desk

A repository owns database operations. UserRepository finds an account by email and inserts a new user. The service asks the repository to do those operations without needing to know the SQL/Drizzle details.

`src/repositories/user-repository.ts`

src/middlewares - checks every request in the hallway

Middleware runs between receiving a request and reaching the controller. The schema middleware validates the request. If invalid, it sends a 400 response; if valid, next() lets it continue.
Analogy: a security/check-in desk stops incomplete forms before they reach staff.

`src/middlewares/schema-middleware.ts`

src/schema - the form rules

auth-schema.ts contains Zod rules for registration, login, and refresh-token request bodies. It also derives TypeScript types from those rules. This is not the same as the database schema.

`src/schema/auth-schema.ts`

src/db - database connection and table design

db/index.ts opens the PostgreSQL connection and creates the Drizzle database object. db/schema.ts describes tables and columns to Drizzle. Analogy: index.ts is the phone line to the records office; schema.ts is the database's blueprint.

`src/db/index.ts`
`src/db/schema.ts`

src/config - settings

env.ts reads settings such as the port and database URL from environment variables. Settings keep environment-specific values out of the application logic.

`src/config/env.ts`

src/app.ts and src/server.ts - build the app and turn it on

app.ts creates the Express application, enables JSON parsing, and mounts the auth router. server.ts starts the app listening on ENV.PORT. Analogy: app.ts arranges the desks; server.ts opens the building for visitors.

`src/app.ts`
`src/server.ts`

Folders and files not yet doing the full auth job

A login schema exists, but the login controller/service/route flow is not implemented yet. A Log-In-Service.ts placeholder exists, but it is not part of registration. Email verification is not implemented. There is no need to delete the database, migrations, or server to build these features later.

3. Foundations: common programming words

Values and data types

- string: text, such as an email or a person's name.
- number: a numeric value, such as 7000 or an HTTP status code.
- boolean: true or false, such as whether an email was found.
- object: named values grouped together, such as { name, email }.
- array: an ordered list of values, such as rows returned by a database query.
- undefined: no value was found or assigned. A lookup can return undefined when there is no
- matching user.
- null: an explicitly empty value. It is different from undefined.

Variables, constants, and names

const creates a variable binding that cannot be reassigned. The object it points to might still be mutable, but simple values such as a string cannot be reassigned to that const name. Names like input, userRepository, and passwordHash are names chosen by the programmer; they are not special words.

Functions, parameters, and returns

A function is a reusable set of steps. A parameter is an input to that function. return sends a result back to its caller and stops that function. Example: signUpService(input) receives signup fields; return sends a success or error result to the controller.

Classes and instances

A class groups related methods and describes how to make an object. UserRepository is a class. new UserRepository() makes an instance that the service can use, like taking a working copy of the records-desk toolkit.

Async and await

A database or password hash takes time. async marks a function that can perform asynchronous work. await pauses that function until the work finishes, without freezing the whole server. The result becomes available after the wait.

Imports and exports

An import brings a value or type from another package/file into the current file. An export makes something available for other files to import. Like borrowing a tool from another room and labeling a tool for others to use.

```
import { z } from "zod";
```

The braces mean a named export is being selected. In this example, the Zod package exports a named value called z. z is Zod's main API object: it provides tools such as z.object(), z.string(), and z.email(). It is not a TypeScript keyword and it was not invented in this file. The package was installed as a project dependency.

```
import type { Request, Response } from "express";
```

The word type means these imports are only needed by TypeScript to check types. They are not runtime objects the app has to call.

Types are not runtime validation

A TypeScript type helps check our code while developing/building. It does not inspect a real HTTP request by itself. Zod's safeParse does runtime validation: it checks the actual data that arrived from a client.

Equality and comparisons

Drizzle's eq helper means equal. eq(users.email, email) asks the database for rows where the email column equals the supplied email. An if statement checks a condition and chooses whether to run its block.

4. The request rules: auth-schema.ts

This file defines the shape that a registration request must follow. The outer body object mirrors Express's req.body location in the request.

```
import { z } from "zod";
```

Line meaning: import the named value z from the Zod library. z is the toolbox used to build validation rules.

```
export const registerSchema = z.object({
```

export makes registerSchema available to the route. const gives the schema a stable variable name. z.object() says the value must be an object with the properties listed inside it.

```
body: z.object({
```

The schema expects an outer object with a property named body, and body itself must be an object. The middleware supplies this wrapper by checking { body: req.body }.

```
name: z.string({ message: "Name is required" }).min(3, ...),
```

name must be text and at least 3 characters long. The message is the explanation if the string requirement fails. Note: this checks length, not whether the name contains only letters.

```
email: z.email({ message: "Email is required and must be a valid email address" }),
```

z.email() checks that the value is text in an email-like format. It does not prove that the mailbox exists or belongs to the user.

```
password: z.string({ message: "Password is required" }).min(10, ...),
```

password must be text and at least 10 characters. This is a minimum-length rule, not a password-strength meter.

```
phoneNumber: z.string({ message: "Phone number is required" }),
countryCode: z.string({ message: "Country code is required" }),
```

Both fields must be strings. As written, an empty string is still a string, so stronger non-empty/format rules could be added later.

```
city: z.string({ message: ... })
.regex(/^[A-Za-z\s]+$/, { message: ... })
.regex(/^[A-Z][a-zA-Z\s]*$/, { message: ... })
```

City must be text containing only ASCII letters/spaces, and must start with an uppercase letter. This also rejects valid real-world city names with accents, apostrophes, or hyphens; that limitation can be reviewed when desired.

```
}),
});
```

These closing braces and parentheses finish the inner body object, the outer schema object, and the z.object() call.

```
export type RegisterInput = z.infer<typeof registerSchema>["body"];
```

export makes a reusable TypeScript type. z.infer<typeof registerSchema> asks TypeScript to derive a type from the Zod schema. ["body"] selects the inner body fields, not the outer wrapper. RegisterInput is a chosen name, not a special word. It represents the six fields and their TypeScript types. This does not validate live data; safeParse does that.

5. The validation checkpoint: schema-middleware.ts

```
import type { RequestHandler } from "express";
```

RequestHandler is Express's TypeScript description for middleware. type means it is only for type checking.

```
import { z } from "zod";
```

The middleware accepts a Zod schema, so it imports z to name that schema type.

```
export function validate(schema: z.ZodType): RequestHandler {
```

Defines and exports a function named validate. Its schema parameter is a Zod schema. It returns an Express RequestHandler. The outer function receives a schema once and creates a middleware function for requests.

```
return (req, res, next) => {
```

Returns the middleware. req is the incoming request, res is how we reply, and next() tells Express to continue to the next handler.

```
const validationResult = schema.safeParse({ body: req.body });
```

safeParse checks the actual request body without throwing for ordinary invalid input. The wrapper matches the schema shape { body: ... }. validationResult contains either success data or validation issues.

```
if (!validationResult.success) {
```

If validation did not succeed, enter the failure block.

```
return res.status(400).json({ status: "error", ... });
```

Send HTTP 400 and a JSON explanation. return is important: it stops the middleware so the controller cannot run for invalid data.

```
}
```

Closes the invalid-input block.

```
next();
```

The data was valid, so hand the request to the next handler: the register controller.

```
};
}
```

The first closing brace ends the returned middleware; the second ends validate.

Middleware order matters

The route registers validate(registerSchema) before register. Express runs handlers in the order listed. Validation is like checking a form for required fields before an account clerk processes it.

6. The database desk: user-repository.ts

```
import { eq } from "drizzle-orm";
```
Imports Drizzle's eq helper. eq is short for equal; it builds a safe database comparison.

```
import { db } from "@/db";
```
Imports the database connection exported by src/db/index.ts. The @ alias is configured to mean src. The /db path resolves through the folder's index.ts.

```
import { users } from "@/db/schema";
```
Imports the Drizzle table definition named users from src/db/schema.ts. It is a description Drizzle uses to build queries, not a copy of the table's rows.

```
export class UserRepository {
```
Defines an exported class that groups user database operations. The service can create an instance with new UserRepository().

```
async findByEmail(email: string) {
```
Defines an async method. It receives one string, the email to search for. Async is needed because the database query takes time.

```
const [user] = await db
.select({ id: users.id })
.from(users)
.where(eq(users.email, email));
```
This chain means: wait for a query; select only the id column; search the users table; keep rows where users.email equals the supplied email. The database returns an array; [user] takes its first row. The email column is unique, so there can be at most one matching account. If there is no match, user is undefined.

```
return user;
```
Give the matching ID result back to the service. The query intentionally does not retrieve the password hash for this duplicate check.

```
}
```
Closes findByEmail.

```
async create(data: typeof users.$inferInsert) {
```
Defines the insert method. data contains the columns to save. typeof users.$inferInsert asks TypeScript for the allowed insert shape based on the Drizzle users table. This prevents spelling a column like password instead of passwordHash.

```
const [newUser] = await db
.insert(users)
.values(data)
.returning({ id: users.id });
```
Insert into users with the supplied values. returning asks PostgreSQL to return a selected result from the inserted row. Only the id is requested, so the password hash and other fields are not returned.

```
return newUser;
```
Give the new ID object to the service. The service wraps it in its result.

```
}
}
```
The final braces close create and the UserRepository class.

Database uniqueness is important

The service first checks for a duplicate email, which gives a friendly response. The database

also has a unique constraint on email, which is essential: two simultaneous requests could both pass the check before either inserts. A production-ready service should translate a unique-constraint error into a clean conflict response.

7. The signup procedure: Sign-Up-Service.ts

```
import bcrypt from "bcrypt";
```
Imports bcrypt's default API. bcrypt makes a one-way password hash. The stored value is not the original password.

```
import type { RegisterInput } from "@/schema/auth-schema";
```
Imports the inferred TypeScript shape for registration data. type means compile-time only. @/schema resolves to src/schema.

```
import { UserRepository } from "@/repositories/user-repository";
```
Imports the records-desk class so the service delegates database work to the repository.

```
export async function signUpService(input: RegisterInput) {
```
Export makes the function available to the controller. async allows database and hashing waits. input is the parameter name; RegisterInput describes its fields. The opening brace starts the procedure.

```
const { name, email, password, phoneNumber, countryCode, city } = input;
```
Object destructuring copies named fields from input into local variables. Instead of writing input.email each time, the function can use email.

```
const userRepository = new UserRepository();
```
Create an instance of the repository. This gives the service methods such as findByEmail and create.

```
const existingUser = await userRepository.findByEmail(email);
```
Ask the repository whether this email is already in the database, and wait for its result.

```
if (existingUser) {
return { code: 400, status: "error", message: "Email already exists" };
}
```
If a match exists, return an error result immediately. return stops the signup procedure, so hashing and inserting do not happen for this ordinary duplicate case. 400 means the request conflicts with current data in this project's chosen response convention; HTTP 409 Conflict is another common choice.

```
const passwordHash = await bcrypt.hash(password, 12);
```
Wait for bcrypt to hash the submitted password. 12 is bcrypt's work factor/cost setting: higher values take more computation. The app saves passwordHash, never the raw password.

```
const newUser = await userRepository.create({
name, email, passwordHash, phoneNumber, countryCode, city,
});
```
Ask the repository to insert the account. The object property shorthand means { name } is the same as { name: name }. passwordHash matches the Drizzle column/property name in db/schema.ts. await waits for the insert result.

```
return {
code: 201, status: "success",
message: "Account created successfully.",
data: { user: newUser },
};
```
Return the success object to the controller. 201 is the HTTP Created status. data.user contains the ID object the repository returned. This is an internal service result until the controller sends it.

```
}
```
Closes the function.

Why the layers matter

The signup service does not construct SQL or choose Express response methods. It coordinates the business steps. The repository owns persistence. The controller owns HTTP communication. This is the same separation used in the instructor's example, adapted to Rootine's six registration fields and passwordHash column.

8. Controller, route, app, and server

auth-controllers.ts

```
import type { Request, Response } from "express";
```
Import Express's request/response types for TypeScript checking; type means these aren't runtime imports.

```
import { signUpService } from "@/services/auth/Sign-Up-Service";
```
Import the signup procedure so the controller can call it.

```
export async function register(req: Request, res: Response) {
```
Export a controller named register. req is typed as an Express request and res as its response. async allows awaiting the service.

```
const result = await signUpService(req.body);
```
Read the parsed JSON body from req.body, pass it to the service, and wait for the service result. The request middleware already checked its shape.

```
return res.status(result.code).json(result);
```
Use the service's code as the HTTP status, then send the result as JSON. return ends the controller after responding.

```
}
```

auth-routes.ts

```
import { Router } from "express";
```
Router is a named Express export used to group routes.

```
import { register } from "@/controllers/auth-controllers";
import { registerSchema } from "@/schema/auth-schema";
import { validate } from "@/middlewares/schema-middleware";
```
These imports bring in the controller, validation rules, and validation middleware.

```
const authRouter = Router();
```
Create a router instance to hold the auth endpoints.

```
authRouter.post("/register", validate(registerSchema), register);
```
Register a POST route at /register. First validate the body with registerSchema; then call register if validation succeeded. app.ts adds the /auth prefix, producing /auth/register.

```
export default authRouter;
```
Make this router the default export so app.ts can import it without braces.

app.ts

```
import express from "express";
import authRouter from "@/routes/auth-routes";
const app = express();
```
Import Express and the router, then create the application.

```
app.use(express.json());
```
Install built-in middleware that parses incoming JSON request bodies into req.body.

```
app.use("/auth", authRouter);
```
Mount every auth route below the /auth path prefix.

```
export default app;
```
Export the configured app for server.ts.

server.ts

```
import app from "./app";
import { ENV } from "@/config/env";
app.listen(ENV.PORT, () => {
console.log(`Server is running on port ${ENV.PORT}`);
});
```

Import the app and environment settings. listen starts accepting network requests on the configured port. The callback prints a startup message. Template-literal backticks allow ${ENV.PORT} to insert the port value.

9. Database blueprint, testing, and what is next

Drizzle database schema vs Zod request schema

Zod's auth-schema.ts checks data coming into the API. Drizzle's db/schema.ts describes database tables and columns. They solve different problems: one checks the submitted form; the other tells the database library how data is stored.

- users has an ID, name, unique email, passwordHash, role, phoneNumber, countryCode, city,
- createdAt, and updatedAt.
- tokens has a token type, token value, expiry/lifecycle dates, and a userId link. The table
- exists, but email verification/token behavior is not implemented yet.
- db/index.ts loads DATABASE_URL, creates a PostgreSQL connection pool, and gives Drizzle the
- schema.
- Migration SQL files describe database changes over time. Do not delete the database or rerun
- migrations blindly. The name column was added by a migration, so database migration status
- matters if setup errors occur.

How the tests were run

The backend was started in one terminal with npm run dev from the backend folder. A second PowerShell terminal sent a JSON POST request to localhost:7000/auth/register.

Observed outcomes: a valid request returned 201; the same email returned 400 with Email already exists; a request missing city returned 400 with the issue path body.city. This confirmed the route, validation, duplicate lookup, and account insert were operating together.

The successful request created a persistent test account. The duplicate test confirmed it was still there. Do not treat the sample test row as temporary; it remains until intentionally handled through a suitable database operation.

Important follow-up before login

- Inspect the actual JSON response for the first successful signup. PowerShell displayed the
- nested user as empty-looking, even though the repository code intends to return an ID.
- Confirm whether the response contains an ID.
- Handle a duplicate-email database unique-constraint race cleanly; the initial lookup alone
- cannot prevent simultaneous requests.
- Normalize email consistently (for example trimming and lowercasing) before both lookup and
- insertion.
- Consider stronger phone/country-code validation and city rules that support
- accented/hyphenated names.
- Add a centralized error response so unexpected database/hash errors do not become confusing
- default server responses.
- Only after this, build login: find the user by email, retrieve the stored password hash for
- comparison, compare with bcrypt, and decide the token/session flow.

Quick glossary

- API: a set of URLs and rules clients use to talk to a server.
- HTTP method: the kind of request; POST commonly submits/creates data.

- JSON: a text format for objects and arrays sent over a network.
- Middleware: a request-processing step that can respond or pass control onward.
- Hash: a one-way transformed value; bcrypt is designed for passwords.
- Repository: a layer that keeps database queries separate from business rules.
- Schema: a description of valid data or database structure; context tells which kind.
- Status code: a number summarizing the HTTP outcome, such as 201 created or 400
- invalid/conflicting input.

End of guide. The database and migrations were not changed while preparing this document.